KUBERNETES

Pods, From Scratch

A beginner's guide — plain-language concepts, real YAML examples, and kubectl commands

First, a Quick Refresher: What Is Kubernetes?

Before Pods make sense, it helps to know the problem Kubernetes solves.

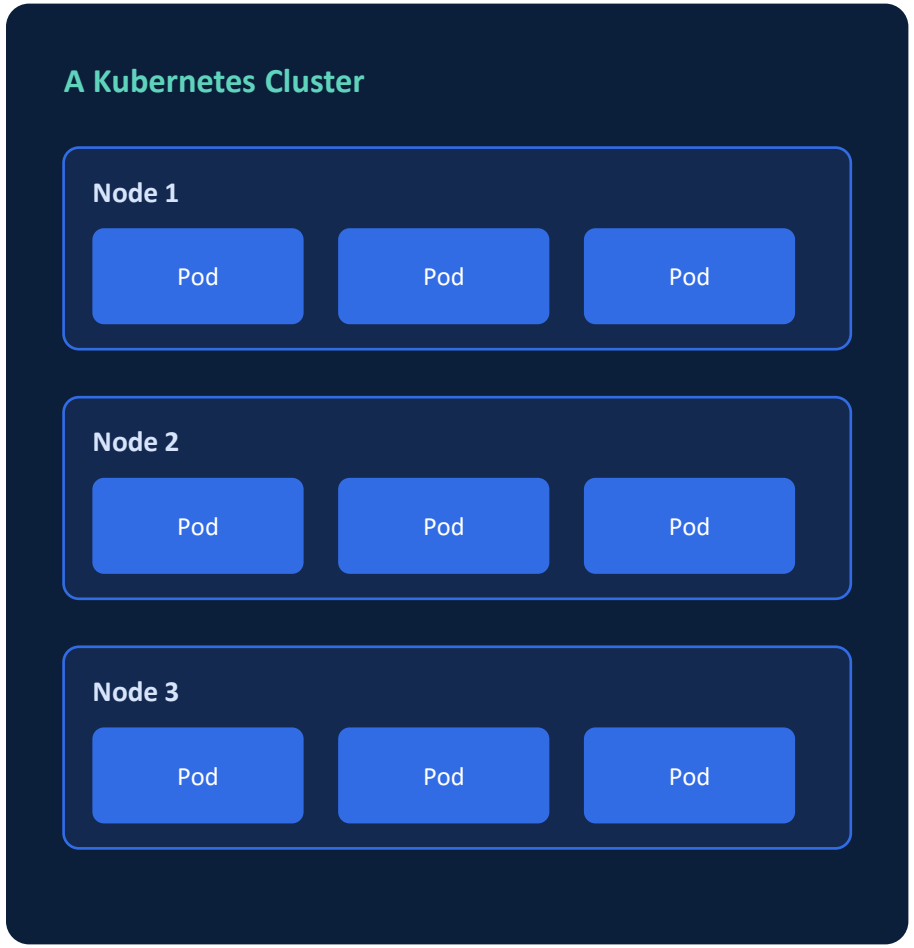
The problem

You've packaged your app into containers. Now you have dozens, maybe thousands, running across many machines. Who decides which machine runs which container? What happens when a machine dies? How do containers find each other?

The answer

Kubernetes is a container orchestrator. You describe what you want ("run 3 copies of this app"), and Kubernetes places, monitors, restarts, and scales your containers across a cluster of machines automatically.

Pods are the unit Kubernetes actually schedules and runs — everything else builds on top of them.



The Simplest Way to Think About a Pod

An analogy before the definition.

An apartment

Roommates sharing an apartment share one street address and one set of utilities (water, electricity). They can walk between rooms freely. If the building is demolished, all roommates are displaced together.

apartment = Pod · address = Pod IP · utilities = shared volumes · roommates = containers

A Kubernetes Pod

Containers grouped into a Pod share one network address (IP) and, optionally, storage volumes. They're always scheduled onto the same node. If the Pod is removed, every container in it goes with it.

A Pod is not a container — it's a thin wrapper that can hold one or more containers, plus the shared resources they use.

What Is a Pod, Formally?

The smallest deployable unit in Kubernetes.

1. **Shared context**

   Containers in a Pod share the same network namespace and, optionally, storage volumes.

2. **Co-scheduled**

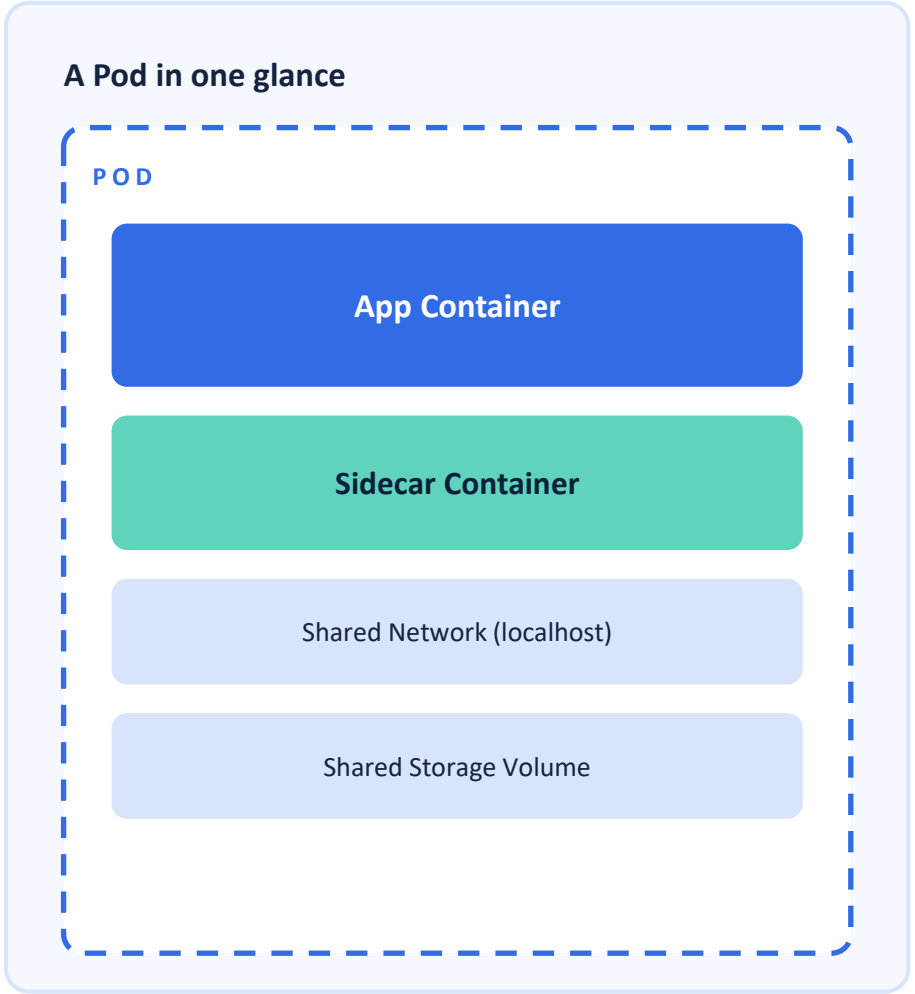
   All containers in a Pod are always placed on the same node and scheduled together.

3. **Single IP address**

   The Pod gets one cluster IP; containers inside it talk to each other over localhost.

4. **Ephemeral by design**

   Pods are disposable — Kubernetes replaces failed Pods rather than repairing them in place.



Why Not Just Run Containers Directly?

Kubernetes never schedules a bare container on its own — it always schedules a Pod.

Without Pods

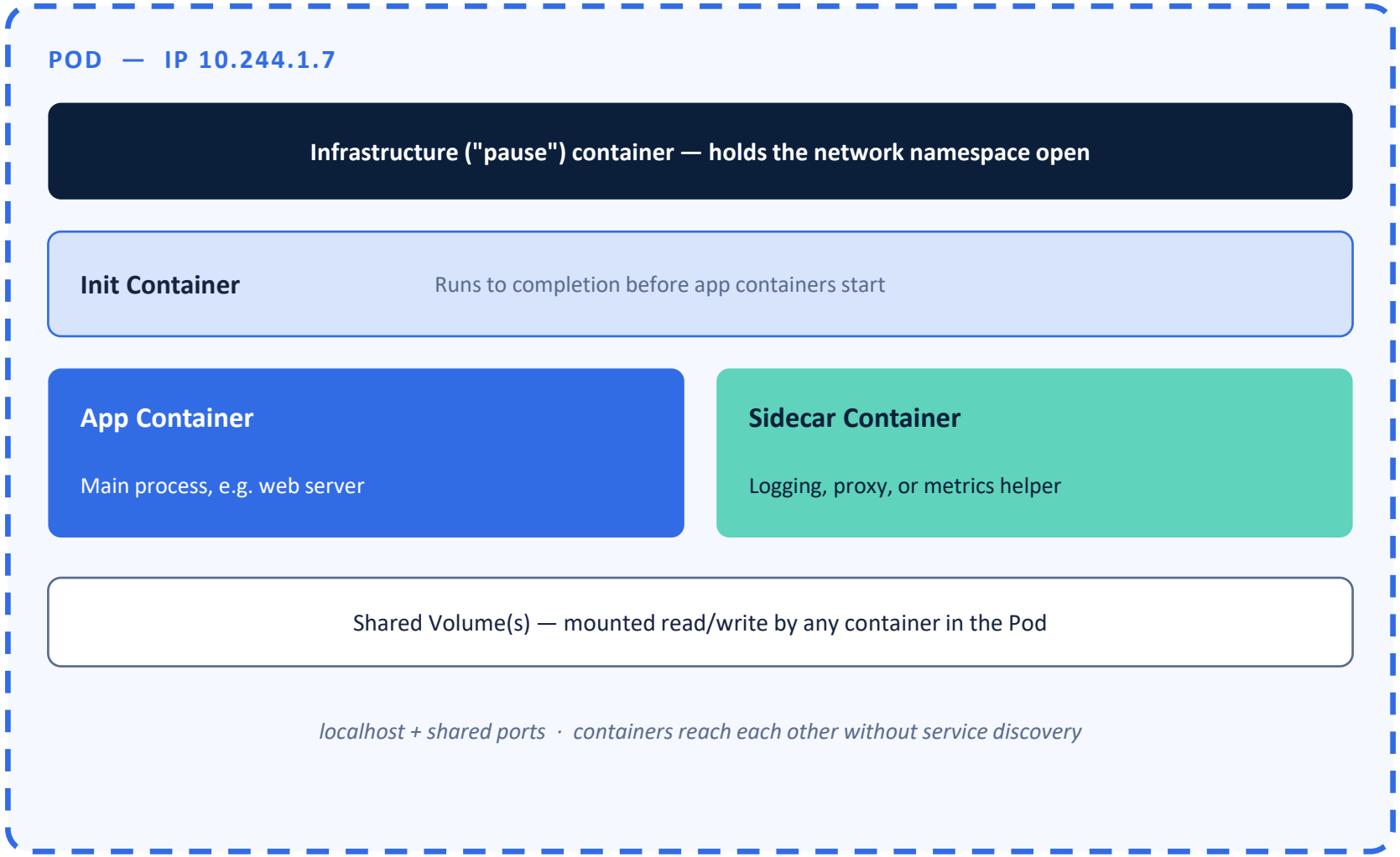
- ✕ No natural way to say "these two containers must always run together."
- ✕ No shared network identity for tightly-coupled helpers (logging, proxying).
- ✕ No shared storage between cooperating containers without extra plumbing.

With Pods

- ✓ The Pod spec explicitly groups containers that belong together.
- ✓ Every container in the Pod shares one IP and can talk over localhost.
- ✓ A Volume declared once in the Pod spec is available to every container in it.

Anatomy of a Pod

Every Pod is wrapped in a single network and storage boundary, no matter how many containers run inside.



Key Facts

- One IP per Pod, shared by all containers
- Containers communicate via localhost
- Volumes outlive container restarts
- Init containers run in order, then exit
- Defined in a single YAML manifest

Example: Your First Pod

A Pod is defined declaratively in YAML — you describe the desired state, Kubernetes makes it real.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-first-pod
spec:
  containers:
    - name: web
      image: nginx:latest
      ports:
        - containerPort: 80
```

`apiVersion / kind`

Tells Kubernetes this document describes a Pod.

`metadata.name`

The Pod's name — how you'll refer to it later.

`spec.containers`

The list of containers to run inside this Pod.

`image`

Which container image to pull and run.

Example: Creating and Viewing a Pod

Once the YAML file exists, kubectl talks to the cluster on your behalf.

```
$ kubectl apply -f pod.yaml
pod/my-first-pod created

$ kubectl get pods
NAME            READY   STATUS    RESTARTS   AGE
my-first-pod    1/1     Running   0          8s

$ kubectl describe pod my-first-pod
...  (full details: events, container status, IP, node)
```

apply

Sends the YAML to Kubernetes; it creates the Pod to match.

get pods

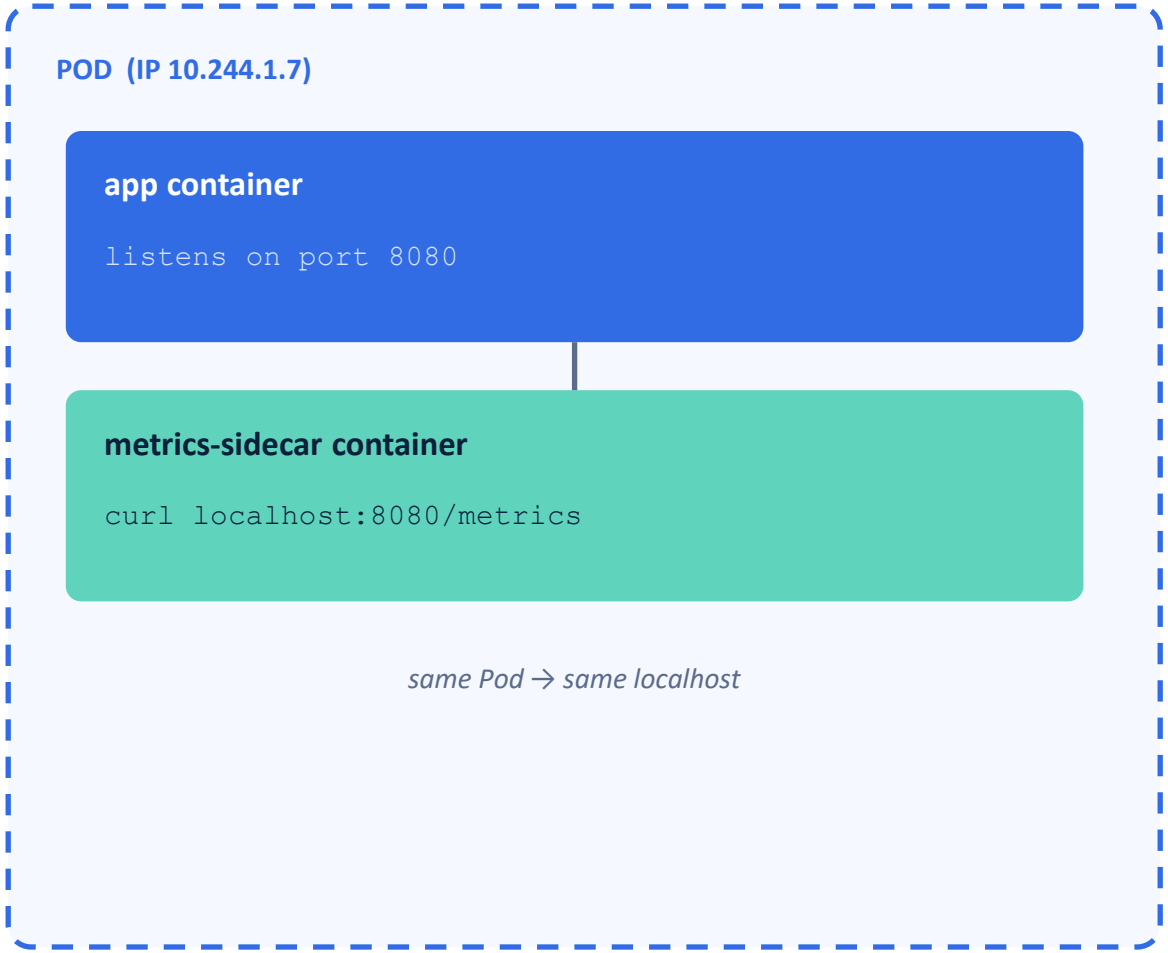
Lists Pods and their current status at a glance.

describe

Shows full detail — scheduling events, restarts, IP address.

Example: How Containers Talk Inside a Pod

Two containers in the same Pod share localhost — no networking setup required.




```
spec:
  containers:
    - name: app
      image: my-app:v1
      ports:
        - containerPort: 8080

    - name: metrics-sidecar
      image: metrics-agent:v1
      # reaches the app via:
      # http://localhost:8080
```

Example: Sharing Files Between Containers

A Volume declared at the Pod level can be mounted by more than one container.

```
spec:
  containers:
    - name: writer
      image: busybox
      volumeMounts:
        - name: shared-data
          mountPath: /data
    - name: reader
      image: busybox
      volumeMounts:
        - name: shared-data
          mountPath: /data
  volumes:
    - name: shared-data
      emptyDir: {}
```

What's happening

- "shared-data" is an emptyDir volume — created fresh when the Pod starts.
- Both containers mount it at /data, so a file the writer creates, the reader can immediately see.
- The volume's contents survive a container restarting, but are deleted when the whole Pod is removed.

Example: A Real Sidecar Pattern

The app container just writes logs to a file — it doesn't know or care who reads them.

```
spec:
  containers:
    - name: app
      image: my-app:v1
      volumeMounts:
        - name: logs
          mountPath: /var/log/app

    - name: log-shipper
      image: fluentd:v1
      volumeMounts:
        - name: logs
          mountPath: /var/log/app

  volumes:
    - name: logs
      emptyDir: {}
```

Why split it this way?

- The app team owns simple code: write logs to a file, nothing else.
- The platform team owns log-shipper and can upgrade or reconfigure it without touching the app.
- Both containers still restart, scale, and get scheduled together, as one Pod.

Example: Waiting for a Dependency

An init container can block the app from starting until a database is reachable.

```
spec:
  initContainers:
    - name: wait-for-db
      image: busybox
      command:
        - sh
        - -c
        - until nc -z db 5432;
          do sleep 2; done

  containers:
    - name: app
      image: my-app:v1
```

How it behaves

- "wait-for-db" starts first and retries the connection check every 2 seconds.
- The "app" container does not start until wait-for-db exits successfully.
- If there were multiple init containers, they'd run one at a time, in order.

Pod Lifecycle

Every Pod moves through a defined set of phases from creation to termination.

Pending

Accepted; waiting to be scheduled and for images to be pulled.

→

Running

Bound to a node; at least one container is running or restarting.

→

Succeeded

All containers terminated successfully and won't be restarted.

→

Failed

All containers terminated, at least one exited with an error.

Unknown — the Pod's state can't be confirmed, usually a lost connection to its node.

Example: watching a Pod move through phases

```
$ kubectl get pod my-first-pod --watch
NAME           READY   STATUS              RESTARTS   AGE
my-first-pod   0/1     Pending             0          1s
my-first-pod   0/1     ContainerCreating   0          2s
my-first-pod   1/1     Running             0          4s
```

Restart Policy & Self-Healing

Kubernetes doesn't repair a broken Pod — it restarts containers according to a policy you set.

Always

Restart no matter how it exits. Default — used for web servers, APIs.

OnFailure

Restart only on a non-zero exit code. Common for batch jobs.

Never

Do not restart after exit, success or failure. One-shot tasks.

```
spec:
  restartPolicy: OnFailure    # Always | OnFailure | Never
```

What "self-healing" actually means

A crashed container restarts in place, on the same Pod, following the policy above — with a growing backoff delay between attempts (CrashLoopBackOff). But if the Pod itself is deleted or its node fails, nothing recreates it: that job belongs to a controller like a Deployment.

Key Takeaways

What

A Pod bundles one or more containers behind one IP and, optionally, shared volumes.

Why

It's Kubernetes' way of saying "these containers belong together, always."

How

Defined in YAML (apiVersion, kind, metadata, spec) and managed via kubectl.

Behaviour

Pods move through Pending → Running → Succeeded/Failed; containers restart per policy.

Beyond Pods

Controllers like Deployments recreate Pods when they're lost — durability lives one layer up.